

Managing Services

Monitoring and controlling services on a host is integral to being an administrator. As an attacker, the ability to interrogate services, find solid points to hook into, and turn services on or off is a sought-after ability when landing on a host. In this section, we will look at the usage of **sc**, the Windows command line service controller utility, but we will go about it a little differently. Let's look at this from the perspective of an attacker. We have just landed on a victims host and need to:

- Determine what services are running.
- Attempt to disable antivirus.
- Modify existing services on a system.

Service Controller

SC is a Windows executable utility that allows us to query, modify, and manage host services locally and over the network. For most of this section, we will utilize **sc** as our defacto way to handle services. We have other tools, like Windows Management Instrumentation (**WMI**) and **Tasklist** that can also query and manage services for local and remote hosts. Let's dive in and give **sc** a try.

SC without Parameters

Managing Services

```
C:\htb> sc
```

DESCRIPTION:

```
SC is a command line program used for communicating with the
Service Control Manager and services.
```

USAGE:

```
sc <server> [command] [service name] <option1> <option2>...
```

```
The option <server> has the form "\\ServerName"
```

```
Further help on commands can be obtained by typing: "sc [command]"
```

Commands:

```
query-----Queries the status for a service, or
               enumerates the status for types of services.
queryex-----Queries the extended status for a service, or
               enumerates the status for types of services.
start-----Starts a service.
pause-----Sends a PAUSE control request to a service.
```

```
<SNIP>
```

SYNTAX EXAMPLES

```
sc query          - Enumerates status for active services & drivers
sc query eventlog - Displays status for the eventlog service
sc queryex eventlog - Displays extended status for the eventlog service
sc query type= driver - Enumerates only active drivers
sc query type= service - Enumerates only Win32 services
sc query state= all - Enumerates all services & drivers
sc query bufsize= 50 - Enumerates with a 50 byte buffer
sc query ri= 14 - Enumerates with resume index = 14
sc queryex group= "" - Enumerates active services not in a group
sc query type= interact - Enumerates all interactive services
sc query type= driver group= NDIS - Enumerates all NDIS drivers
```

```
[us-academy-1]-[10.10.14.155]-[htb-ac-819475@htb-byk9ife5jk]-[~]  
[★]$
```

Query Services

Being able to **query** services for information such as the **process state**, **process id (pid)**, and **service type** is a valuable tool to have in our arsenal as an attacker. We can use this to check if certain services are running or check all existing services and drivers on the system for further information. Before we look specifically into checking the Windows Defender service, let's see what services are currently actively running on the system. We can do so by issuing the following command: **sc query type= service**.

Note: The spacing for the optional query parameters is crucial. For example, **type= service**, **type=service**, and **type =service** are completely different ways of spacing this parameter. However, only **type= service** is correct in this case.

Query All Active Services

Managing Services

```
C:\htb> sc query type= service
```

```
SERVICE_NAME: Appinfo  
DISPLAY_NAME: Application Information  
        TYPE           : 30  WIN32  
        STATE           : 4  RUNNING  
                        (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)  
        WIN32_EXIT_CODE  : 0  (0x0)  
        SERVICE_EXIT_CODE : 0  (0x0)  
        CHECKPOINT       : 0x0  
        WAIT_HINT        : 0x0
```

```
SERVICE_NAME: AppXSvc  
DISPLAY_NAME: AppX Deployment Service (AppXSVC)  
        TYPE           : 30  WIN32  
        STATE           : 4  RUNNING  
                        (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)  
        WIN32_EXIT_CODE  : 0  (0x0)  
        SERVICE_EXIT_CODE : 0  (0x0)  
        CHECKPOINT       : 0x0  
        WAIT_HINT        : 0x0
```

```
SERVICE_NAME: AudioEndpointBuilder  
DISPLAY_NAME: Windows Audio Endpoint Builder  
        TYPE           : 30  WIN32  
        STATE           : 4  RUNNING  
                        (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)  
        WIN32_EXIT_CODE  : 0  (0x0)  
        SERVICE_EXIT_CODE : 0  (0x0)  
        CHECKPOINT       : 0x0  
        WAIT_HINT        : 0x0
```

```
SERVICE_NAME: Audiosrv  
DISPLAY_NAME: Windows Audio  
        TYPE           : 10  WIN32_OWN_PROCESS  
        STATE           : 4  RUNNING  
                        (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)  
        WIN32_EXIT_CODE  : 0  (0x0)  
        SERVICE_EXIT_CODE : 0  (0x0)  
        CHECKPOINT       : 0x0  
        WAIT_HINT        : 0x0
```

```
SERVICE_NAME: BFE  
DISPLAY_NAME: Base Filtering Engine  
        TYPE           : 20  WIN32_SHARE_PROCESS  
        STATE           : 4  RUNNING  
                        (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)  
        WIN32_EXIT_CODE  : 0  (0x0)  
        SERVICE_EXIT_CODE : 0  (0x0)
```

Integrated Terminal

```
SERVICE_NAME: BITS
DISPLAY_NAME: Background Intelligent Transfer Service
        TYPE           : 30  WIN32
        STATE            : 4  RUNNING
                        (STOPPABLE, NOT_PAUSABLE, ACCEPTS_PRESHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE : 0  (0x0)
        CHECKPOINT        : 0x0
        WAIT_HINT         : 0x0
```

<SNIP>

We can see a complete list of the actively running services on this system. Using this information, we can thoroughly scope out what is running on the system and look for anything that we wish to disable or in some cases, services that we can attempt to take over for our own purposes, whether it be for escalation or persistence.

Returning to our scenario, we recently landed on a host and need to **query** the host and determine if Windows Defender is active.

Let's give **sc query** a try.

Querying for Windows Defender

Managing Services

```
C:\htb> sc query windefend

SERVICE_NAME: windefend
        TYPE           : 10  WIN32_OWN_PROCESS
        STATE            : 4  RUNNING
                        (NOT_STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE : 0  (0x0)
        CHECKPOINT        : 0x0
        WAIT_HINT         : 0x0
```

Now, what do we see above? We can tell that Windows Defender is running and, with our current permission set (the one in which we utilized for the query), does not have permission to stop or pause the service (likely because our user is a standard user and not an administrator). We can test this by trying to stop the service.

Stopping and Starting Services

Stopping an Elevated Service

Managing Services

```
C:\htb> sc stop windefend

Access is denied.
```

As we can see from the output above, our current user doesn't have the proper permissions to stop or pause this particular service. To perform this action, we would likely need the permissions of an Administrator account, and in some cases, certain services can only be handled by the system itself. Ideally, attempting to stop an elevated service like this is not the best way of testing permissions, as this will likely lead to us getting caught due to the traffic that will be kicked up from running a command like this.

Now that we've attempted and failed to stop the **windefend** service under a user with standard permissions let us showcase what

step services and the `sc stop service_name` command. Let's try the previous example once again with elevated permissions as the **Administrator** user.

Stopping an Elevated Service as Administrator

Managing Services

```
C:\WINDOWS\system32> sc stop windefend

Access is denied.
```

It seems we still do not have the proper access to stop this service in particular. This is a good lesson for us to learn, as certain processes are protected under stricter access requirements than what local administrator accounts have. In this scenario, the only thing that can stop and start the Defender service is the **SYSTEM** machine account.

As an attacker, learning the restrictions behind what certain accounts have access or lack of access to is very important because blindly trying to stop services will fill the logs with errors and trigger any alerts showing that a user with insufficient privileges is trying to access a protected process on the system. This will catch the blue team's attention to our activities and begin a triage attempt to kick us off the system and lock us out permanently.

Stopping Services

Moving on, let's find ourselves a service we can take out as an Administrator. The good news is that we can stop the Print Spooler service. Let's try to do so.

Finding the Print Spooler Service

Managing Services

```
C:\WINDOWS\system32> sc query Spooler

SERVICE_NAME: Spooler
        TYPE               : 110  WIN32_OWN_PROCESS (interactive)
        STATE                : 4    RUNNING
                                (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0    (0x0)
        SERVICE_EXIT_CODE   : 0    (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0
```

As we can see from the output above, the **Spooler** service is actively running on our current system.

Stopping the Print Spooler Service

Managing Services

```
C:\WINDOWS\system32> sc stop Spooler

SERVICE_NAME: Spooler
        TYPE               : 110  WIN32_OWN_PROCESS (interactive)
        STATE                : 3    STOP_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0    (0x0)
        SERVICE_EXIT_CODE   : 0    (0x0)
        CHECKPOINT           : 0x3
        WAIT_HINT            : 0x4e20

C:\WINDOWS\system32> sc query Spooler
```

Integrated Terminal

```
TYPE           : 110  WIN32_OWN_PROCESS (interactive)
STATE          : 1    STOPPED
WIN32_EXIT_CODE : 0    (0x0)
SERVICE_EXIT_CODE : 0    (0x0)
CHECKPOINT     : 0x0
WAIT_HINT      : 0x0
```

As stated above, we can issue the command `sc stop Spooler` to have Windows issue a `STOP` control request to the service. It is important to note that not all services will respond to these requests, regardless of our permissions, especially if other running programs and services depend on the service we are attempting to stop.

Starting Services

Much like stopping services, we are also able to start services as well. Although stopping services seems to offer a bit more practicality at first to the red team, being able to start services can lend itself to be especially useful in conjunction with being able to modify existing services.

Starting from our previous example, we are still working with the `Spooler` service that was stopped previously. We can restart this service by issuing the `sc start Spooler` command. Let's try it now.

Starting the Print Spooler Service

Managing Services

```
C:\WINDOWS\system32> sc start Spooler
```

```
SERVICE_NAME: Spooler
```

```
TYPE           : 110  WIN32_OWN_PROCESS (interactive)
STATE          : 2    START_PENDING
                  (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
WIN32_EXIT_CODE : 0    (0x0)
SERVICE_EXIT_CODE : 0    (0x0)
CHECKPOINT     : 0x0
WAIT_HINT      : 0x7d0
PID            : 34908
FLAGS          :
```

```
C:\WINDOWS\system32> sc query Spooler
```

```
SERVICE_NAME: Spooler
```

```
TYPE           : 110  WIN32_OWN_PROCESS (interactive)
STATE          : 4    RUNNING
                  (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
WIN32_EXIT_CODE : 0    (0x0)
SERVICE_EXIT_CODE : 0    (0x0)
CHECKPOINT     : 0x0
WAIT_HINT      : 0x0
```

We can see here that upon issuing a start request to the `Spooler` service, we can see that it begins in a `START_PENDING` state and, after another query, is fully up and operational. Typically services will take a few seconds or so to initialize after a request to start is issued.

Modifying Services

In addition to being able to start and stop services, we can also attempt to modify existing services as well. This is where attackers can thrive as we try to modify existing services to serve whatever purpose we need them to. In some cases, we can change them to

of the actions we can take. With such a versatile command, we have many options for manipulating services to do whatever we need them to. Let's go ahead and see if we can modify some services to prevent Windows from updating itself.

Disabling Windows Updates Using SC

To configure services, we must use the `config` parameter in `sc`. This will allow us to modify the values of existing services, regardless if they are currently running or not. All changes made with this command are reflected in the Windows registry as well as the database for Service Control Manager (SCM). Remember that all changes to existing services will only fully update after restarting the service.

Note: It is important to be aware that modifying existing services can effectively take them out permanently as any changes made are recorded and saved in the registry, which can persist on reboot. Please exercise caution when modifying services in this manner.

With all this information out of the way, let's try to take out Windows Updates for our current compromised host.

Unfortunately, the Windows Update feature (Version 10 and above) does not just rely on one service to perform its functionality.

Windows updates rely on the following services:

Service	Display Name
<code>wuauclt</code>	Windows Update Service
<code>bits</code>	Background Intelligent Transfer Service

Let's query all of the required services and see what is currently running and needs to be stopped before making our required changes.

Important: The scenario below requires access to a privileged account. Making updates to services will typically require a set of higher permissions than a regular user will have access to.

Checking the State of the Required Services

```
Managing Services

C:\WINDOWS\system32> sc query wuauclt

SERVICE_NAME: wuauclt
        TYPE               : 30  WIN32
        STATE                : 1  STOPPED
        WIN32_EXIT_CODE       : 0  (0x0)
        SERVICE_EXIT_CODE    : 0  (0x0)
        CHECKPOINT            : 0x0
        WAIT_HINT             : 0x0

C:\WINDOWS\system32> sc query bits

SERVICE_NAME: bits
        TYPE               : 30  WIN32
        STATE                : 4  RUNNING
                        (STOPPABLE, NOT_PAUSABLE, ACCEPTS_PRESHUTDOWN)
        WIN32_EXIT_CODE       : 0  (0x0)
        SERVICE_EXIT_CODE    : 0  (0x0)
        CHECKPOINT            : 0x0
        WAIT_HINT             : 0x0
```

From the information provided above, we can see that the `wuauclt` service is not currently active as the system is not currently in the process of updating. However, the `bits` service (required to download updates) is currently running on our system. We can issue a

Stopping BITS

Managing Services

```
C:\WINDOWS\system32> sc stop bits

SERVICE_NAME: bits
        TYPE               : 30  WIN32
        STATE                : 3   STOP_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0   (0x0)
        SERVICE_EXIT_CODE   : 0   (0x0)
        CHECKPOINT           : 0x1
        WAIT_HINT            : 0x0
```

After ensuring that both services are currently stopped, we can modify the **start type** of both services. We can issue this change by performing the following:

Disabling Windows Update Service

Managing Services

```
C:\WINDOWS\system32> sc config wuauserv start= disabled

[SC] ChangeServiceConfig SUCCESS
```

Disabling Background Intelligent Transfer Service

Managing Services

```
C:\WINDOWS\system32> sc config bits start= disabled

[SC] ChangeServiceConfig SUCCESS
```

We can see the confirmation that both services have been modified successfully. This means that when both services attempt to start, they will be unable to as they are currently disabled. As previously mentioned, this change will persist upon reboot, meaning that when the system attempts to check for updates or update itself, it cannot do so because both services will remain disabled. We can verify that both services are indeed disabled by attempting to start them.

Verifying Services are Disabled

Managing Services

```
C:\WINDOWS\system32> sc start wuauserv

[SC] StartService FAILED 1058:

The service cannot be started, either because it is disabled or because it has no enabled devices associated with it.

C:\WINDOWS\system32> sc start bits

[SC] StartService FAILED 1058:

The service cannot be started, either because it is disabled or because it has no enabled devices associated with it.
```

appropriately.

We have verified that both services are now disabled, as we cannot start them manually. Due to the changes made here, Windows cannot utilize its updating feature to provide any system or security updates. This can be very beneficial to an attacker to ensure that a system can remain out of date and not retrieve any updates that would inhibit the usage of certain exploits on a target system. Be aware that by doing this in this manner, we will likely be triggering alerts for this sort of action set up by the resident blue team. This method is not quiet and does require elevated permissions in a lot of cases to perform.

Other Routes to Query Services

During the course of this section, we have only focused on using `sc` to query, start, stop, and modify services. However, we have other choices regarding how to accomplish some of these same tasks using different commands. In this section, we will strictly focus on using some of these other commands to help with our enumeration by being able to query services and display the available information in different ways.

Using Tasklist

`Tasklist` is a command line tool that gives us a list of currently running processes on a local or remote host. However, we can utilize the `/svc` parameter to provide a list of services running under each process on the system. Let's look at some of the output this can provide.

Managing Services		
C:\htb> tasklist /svc		
Image Name	PID	Services
=====	=====	=====
System Idle Process	0	N/A
System	4	N/A
Registry	108	N/A
smss.exe	412	N/A
csrss.exe	612	N/A
wininit.exe	684	N/A
csrss.exe	708	N/A
services.exe	768	N/A
lsass.exe	796	KeyIso, SamSs, VaultSvc
winlogon.exe	856	N/A
svchost.exe	984	BrokerInfrastructure, DcomLaunch, PlugPlay, Power, SystemEventsBroker
fontdrvhost.exe	1012	N/A
fontdrvhost.exe	1020	N/A
svchost.exe	616	RpcEptMapper, RpcSs
svchost.exe	996	LSM
dwm.exe	1068	N/A
svchost.exe	1236	CoreMessagingRegistrar
svchost.exe	1244	lmhosts
svchost.exe	1324	NcbService
svchost.exe	1332	TimeBrokerSvc
svchost.exe	1352	Schedule
<SNIP>		

As we can see, we have a full listing of processes that are currently running on the system, their respective `PID`, and what service(s) are hosted under each process. This can be very helpful in quickly locating what process hosts what service(s).

Using Net Start

`Net start` is a very simple command that will allow us to quickly list all of the current running services on a system. In addition to `net`

service afterward and be able to perform the actions specified in the command against the service that we provide.

Managing Services

```
C:\htb> net start
```

These Windows services are started:

```
Application Information
AppX Deployment Service (AppXSVC)
AVCTP service
Background Tasks Infrastructure Service
Base Filtering Engine
BcastDVRUserService_3321a
Capability Access Manager Service
cbdhsvc_3321a
CDPUserSvc_3321a
Client License Service (ClipSVC)
CNG Key Isolation
COM+ Event System
COM+ System Application
Connected Devices Platform Service
Connected User Experiences and Telemetry
CoreMessaging
Credential Manager
Cryptographic Services
Data Usage
DCOM Server Process Launcher
Delivery Optimization
Device Association Service
DHCP Client
<SNIP>
```

From the output above, we can see that using `net start` without specifying a `service` will list all of the active services on the system.

Using WMIC

Last but not least, we have `WMIC`. The Windows Management Instrumentation Command (`WMIC`) allows us to retrieve a vast range of information from our local host or host(s) across the network. The versatility of this command is wide in that it allows for pulling such a wide arrangement of information. However, we will only be going over a very small subset of the functionality provided by the `SERVICE` component residing inside this application.

To list all services existing on our system and information on them, we can issue the following command: `wmic service list brief`.

Managing Services

```
C:\htb> wmic service list brief
```

ExitCode	Name	ProcessId	StartMode	State	Status
1077	AJRouter	0	Manual	Stopped	OK
1077	ALG	0	Manual	Stopped	OK
1077	AppIDSvc	0	Manual	Stopped	OK
0	Appinfo	5016	Manual	Running	OK
1077	AppMgmt	0	Manual	Stopped	OK
1077	AppReadiness	0	Manual	Stopped	OK
1077	AppVClient	0	Disabled	Stopped	OK
0	AppXSvc	9996	Manual	Running	OK
1077	AssignedAccessManagerSvc	0	Manual	Stopped	OK
0	AudioEndpointBuilder	2076	Auto	Running	OK
0	Audiosrv	2332	Auto	Running	OK
1077	autotimesvc	0	Manual	Stopped	OK
1077	AxInstSV	0	Manual	Stopped	OK
1077	BDESVC	0	Manual	Stopped	OK
0	BFE	2696	Auto	Running	OK
0	BITS	0	Manual	Stopped	OK

Integrated Terminal

```
0      BthAvctpSvc      4448      Manual      Running      OK
1077   bthserv         0         Manual      Stopped      OK
0      camsvc          5676      Manual      Running      OK
0      CDPSvc          4724      Auto        Running      OK
1077   CertPropSvc     0         Manual      Stopped      OK
0      ClipSVC         9156      Manual      Running      OK
1077   cloudidsvc      0         Manual      Stopped      OK
0      COMSysApp       3668      Manual      Running      OK
0      CoreMessagingRegistrar 1236      Auto        Running      OK
0      CryptSvc        2844      Auto        Running      OK
<SNIP>
```

After doing so, we can see that we have a nice list containing important information such as the **Name**, **ProcessID**, **StartMode**, **State**, and **Status** of every service on the system, regardless of whether or not it is currently running.

Note: It is important to be aware that the **WMIC** command-line utility is currently deprecated as of the current Windows version. As such, it is advised against relying upon using the utility in most situations. You can find further information regarding this change by following this [link](#).

Moving On

As penetration testers, we will constantly interact with Windows services. Since we will not always have GUI access to a host on which we are trying to escalate privileges, we need to understand how to work with services via the command line in various ways. In a later section, we walk through the PowerShell equivalents for the commands shown in this section and show a more blue team approach to working with and monitoring services. Now that we've finished talking about working with services via cmd.exe, let's dive into the all-important topic of Windows Scheduled Tasks.

VPN Servers

⚠ Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

● UDP 1337 ● TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

40ms ▼

Terminate Pwnbox to switch location

Integrated Terminal

 Full Screen

 Terminate

 Reset

Life Left: -1m 

Disconnected

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

 Cheat Sheet Download VPN
Connection File

+ 0  What command string will stop a service named 'red-light'? (full command as the answer)

Submit your answer here...

 Submit Hint

+ 0  What Windows executable will allow us to create, query, and modify services on a host?

Submit your answer here...

 Submit Hint PreviousNext  Cheat Sheet Resources Go to Questions

Table of Contents

Integrated Terminal

CMD

-  Command Prompt Basics 
-  Getting Help 
-  System Navigation 
-  Working with Directories and Files 
-  Gathering System Information 
-  Finding Files and Directories 
-  Environment Variables 
-  [Managing Services](#)
-  Working With Scheduled Tasks

PowerShell

-  CMD Vs. PowerShell
-  All About Cmdlets and Modules
-  User and Group Management
-  Working with Files and Directories
-  Finding & Filtering Content
-  Working with Services
-  Working with the Registry
-  Working with the Windows Event Log
-  Networking Management from The CLI
-  Interacting with The Web
-  PowerShell Scripting and Automation

Finish Strong

-  Skills Assessment

Beyond This Module

My Workstation

 Interact

 Terminate

 Reset

Life Left: -1m

